

Лабораторная работа по ЛАиВМ №2

Отчет по лабораторной работе №2 по ЛАиВМ

Однослойный перцептрон: реализация, обучение и анализ

Выполнил(а): Пирмагомедов Марат Нурутдинович

Группа: Б25-527

Ссылка на GitHub: https://github.com/mvrvvt/LinAlg_2nd_laboratory_perceptron.git



1. Цель работы

1. Реализовать алгоритм обучения однослойного перцептрона с нуля без использования готовых библиотек глубокого обучения.
2. Изучить влияние параметров (скорость обучения, количество эпох, размер батча) на процесс обучения.
3. Ознакомиться с этапами подготовки данных (стандартизация, стратификация).
4. Провести анализ сходимости и сделать выводы.



2. Теоретическое введение

2.1. Модель перцептрона

Перцептрон — простейшая модель искусственного нейрона, предназначенная для бинарной классификации. Модель вычисляет взвешенную сумму входных признаков, добавляет смещение (bias) и пропускает результат через нелинейную функцию активации:

$$z = w^T x + b = \sum_{i=1}^d w_i x_i + b$$

Где $x \in \mathbb{R}^d$ — вектор признаков, $w \in \mathbb{R}^d$ — вектор весов, b — смещение.

В качестве функции активации используется [сигмоида](#), которая переводит выход модели в диапазон $(0, 1)$, позволяя интерпретировать его как вероятность:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Производная сигмоиды выражается через саму функцию, что делает её удобной для градиентного спуска:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

2.2. Функция потерь и градиентный спуск

Для бинарной классификации используется бинарная кросс-энтропия (Log-Loss). Для батча из m примеров:

$$L = -\frac{1}{m} \sum_{i=1}^m [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Обучение заключается в поиске параметров w и b , минимизирующих L . Обновление параметров происходит против антиградиента:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

где η — скорость обучения (learning rate).

Градиенты для кросс-энтропии принимают следующий вид:

$$\frac{\partial L}{\partial w} = \frac{1}{m} X^T (\hat{y} - y), \quad \frac{\partial L}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)$$

»

3. Описание реализации

Для выполнения работы данные были сгенерированы с помощью [make_classification](#) из библиотеки [scikit-learn](#) (500 примеров, 2 признака).

Разделение на обучающую (70%) и тестовую выборки (30%) производилось с использованием стратификации (параметр [stratify=y](#)).

Перед подачей данных в модель применялась [Z-нормализация](#) (стандартизация):

$$x' = \frac{x - \mu}{\sigma}$$

Параметры μ и σ рассчитывались исключительно на обучающей выборке, чтобы избежать утечки данных (data leakage).

Класс `Perceptron` был реализован с использованием библиотеки `NumPy` и включает в себя методы `forward` (прямой проход), `compute_loss` (подсчет ошибки) и `fit` (обучение мини-батчами).

4. Результаты обучения базовой модели

Обучение базовой модели проводилось с параметрами:

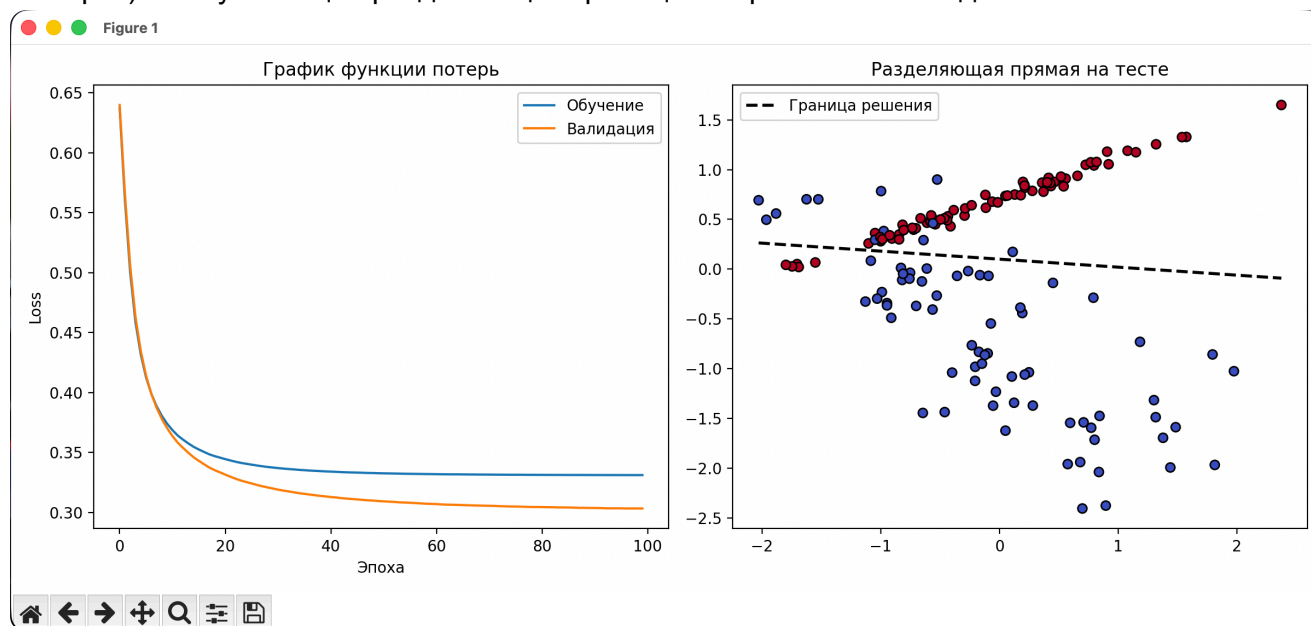
- $\eta = 0.1$
- Количество эпох = 100
- Размер батча = 32

Полученные метрики:

- Accuracy (Train): ~ 0.8686
- Accuracy (Test): ~ 0.8867

Графики базовой модели

Ниже представлены график сходимости функции потерь (обучающая и валидационная выборки) и визуализация разделяющей границы на фоне тестовых данных.

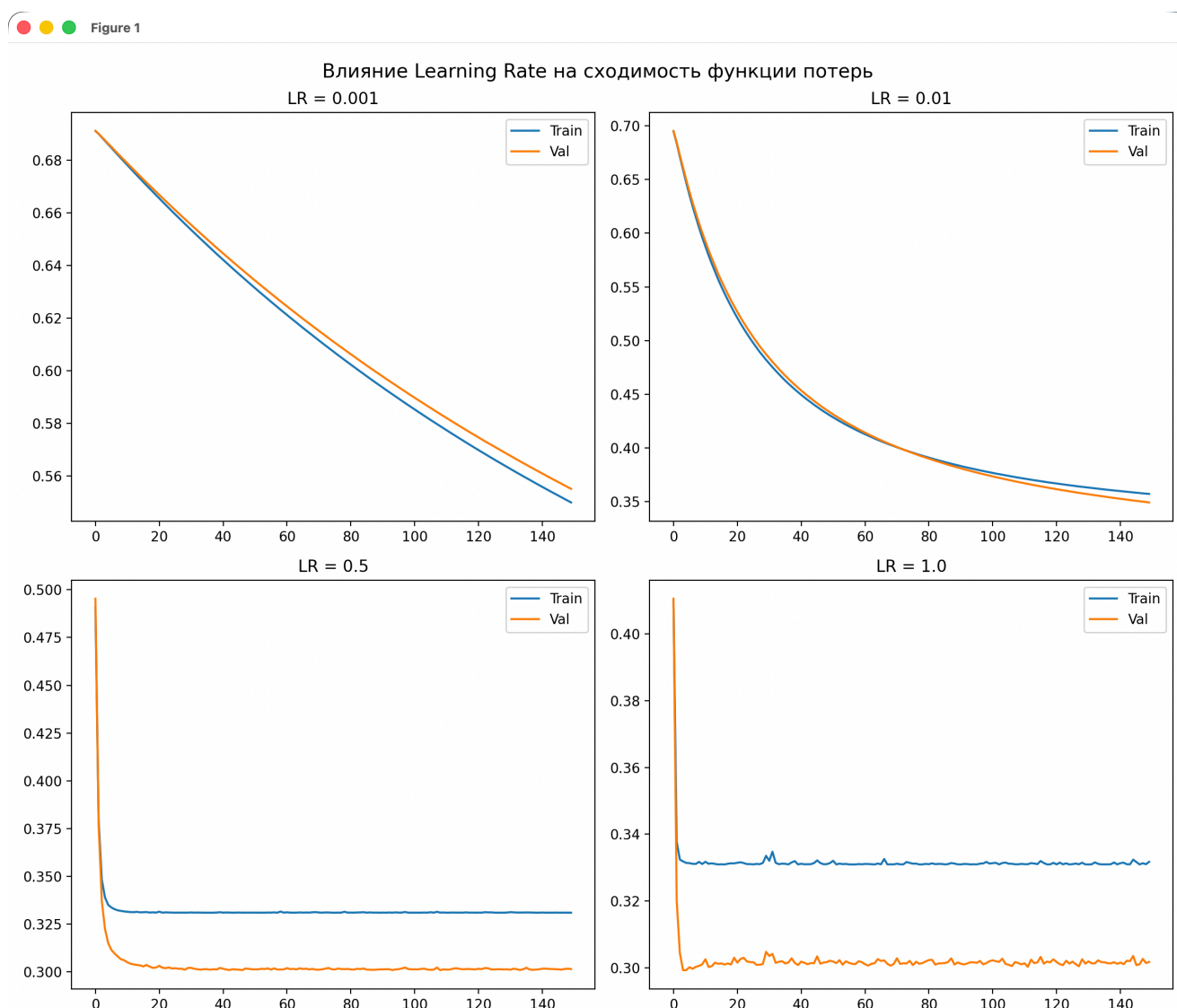


5. Эксперименты и анализ гиперпараметров

Все эксперименты проводились с изоляцией одного параметра (остальные фиксировались на базовых значениях).

5.1. Влияние скорости обучения (Learning Rate)

Скорость обучения (η)	Точность на Train	Точность на Test
0.001	0.8600	0.8800
0.01	0.8600	0.8867
0.1 (Базовая)	0.8686	0.8867
0.5	0.8657	0.8867
1.0	0.8657	0.8867

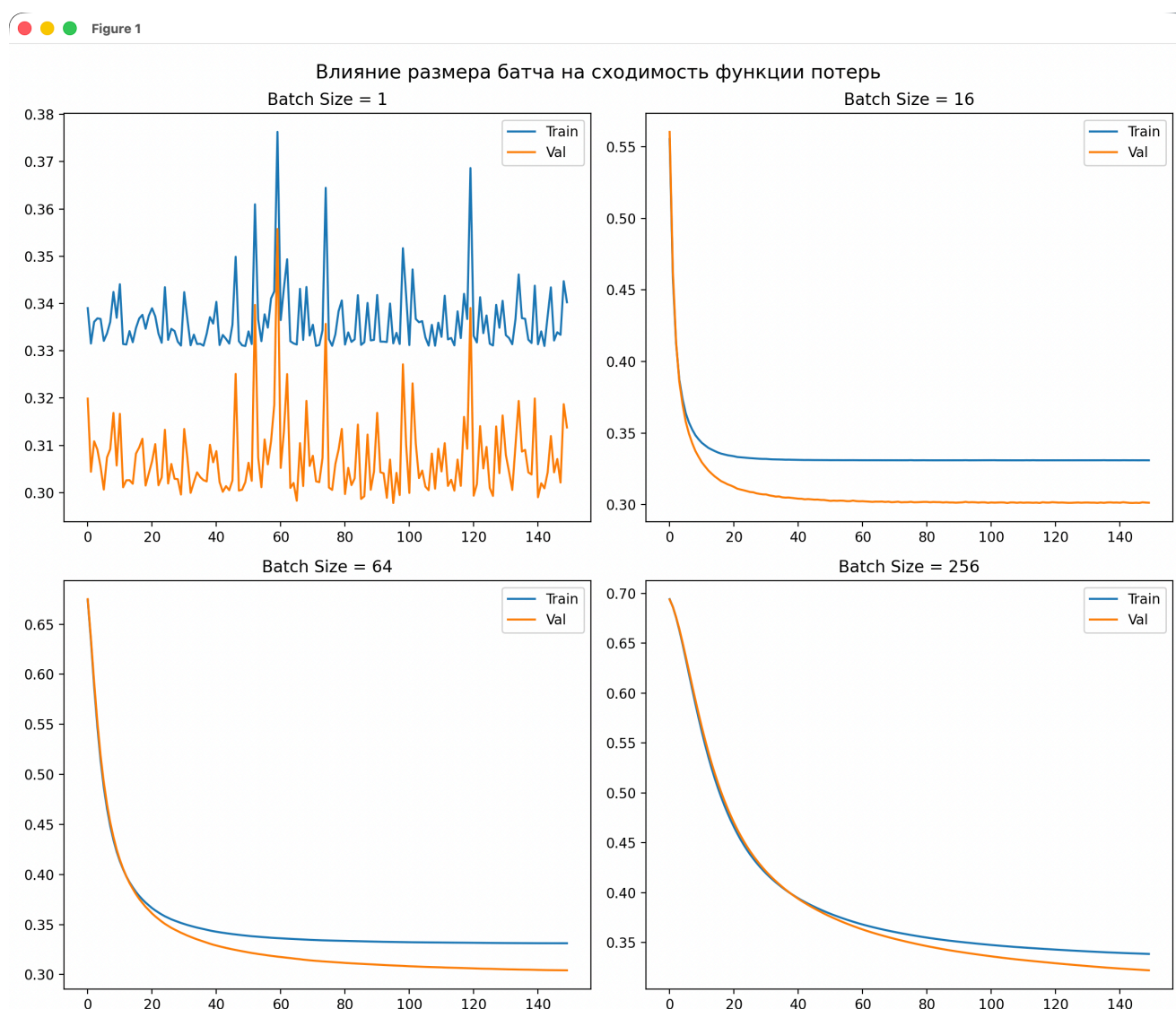


Вывод: При малых значениях η (0.001) модель сходится слишком медленно, и за 150 эпох не успевает достичь оптимума. При слишком больших (1.0) значениях могут наблюдаться

осцилляции, график потерь становится нестабильным («шумным»). Значение $\eta = 0.01$ и $\eta = 0.1$ обеспечивают оптимальную и стабильную сходимость.

5.2. Влияние размера батча (Batch Size)

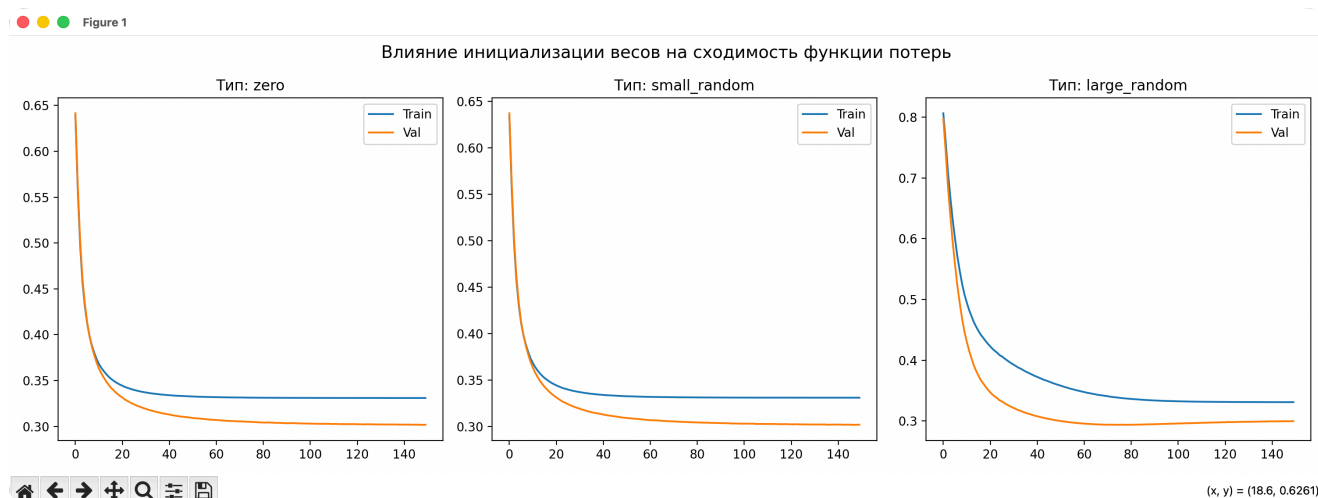
Размер батча	Точность на Train	Точность на Test
1 (SGD)	0.8686	0.8867
16	0.8657	0.8867
64	0.8686	0.8867
256	0.8657	0.8867



Вывод: Обучение по одному примеру (размер 1) дает крайне шумный градиент, из-за чего график loss сильно «дергается». Обучение полным батчем (256) делает градиент точным, но требует больше эпох для сходимости. Мини-батчи (16, 32, 64) представляют собой лучший компромисс между скоростью и стабильностью.

5.3. Влияние инициализации весов

Тип инициализации	Точность на Train	Точность на Test
Zero (нули)	0.8657	0.8867
Small Random	0.8657	0.8867
Large Random	0.8657	0.8867

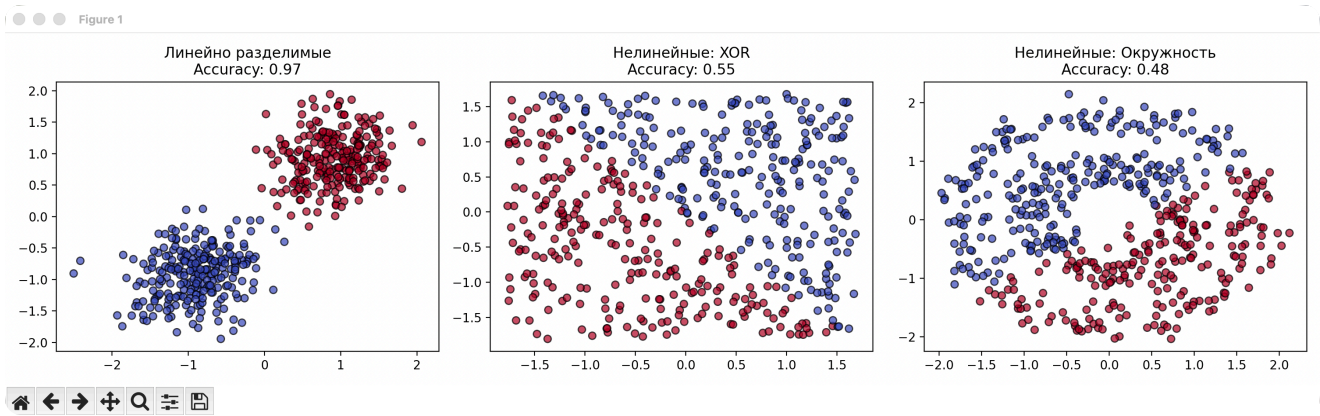


Вывод: Для однослойного перцептрона инициализация нулями допустима, поскольку нет проблемы симметрии, как в многослойных сетях. Однако при инициализации большими весами (`large_random`) модель в начале имеет очень высокие значения потерь, и требуется время на «спуск» к оптимуму, что сильно замедляет первоначальное обучение.

6. Задания на дополнительные баллы (Бонусный блок)

6.1. Собственный генератор данных и границы применимости

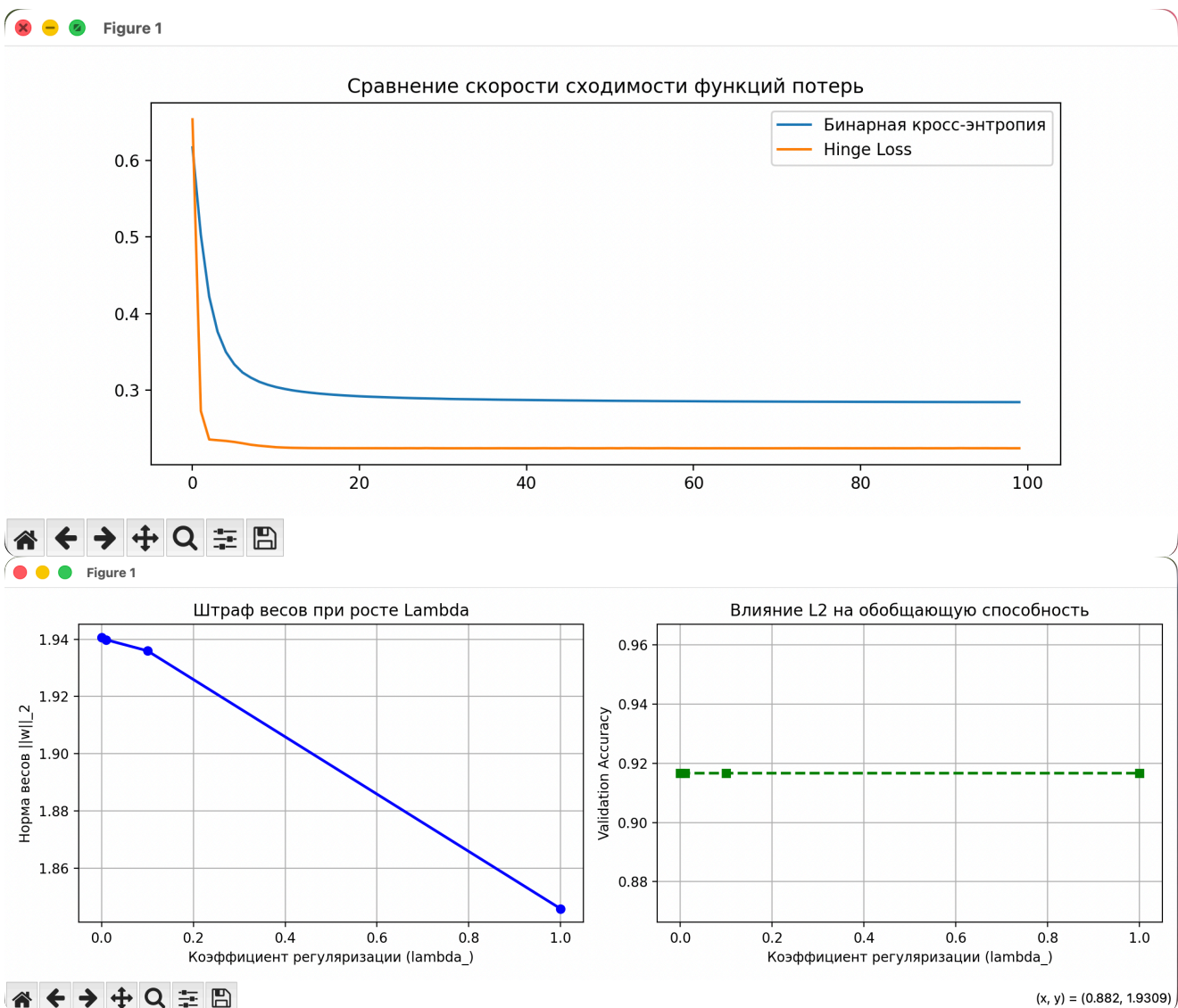
Был написан собственный генератор для создания линейно-разделимых данных, XOR-выборки и структуры «Окружности».



Вывод: Перцептрон успешно нашел разделяющую гиперплоскость (прямую) для линейно разделимых облаков точек. Однако на выборках «XOR» и «Окружности» модель полностью провалилась (Ассигасу ≈ 0.5). Это наглядно доказывает математическое ограничение однослойного перцептрона: он не способен решать линейно неразделимые задачи без дополнительных полиномиальных признаков или скрытых слоев.

6.2. Hinge Loss и L2-регуляризация

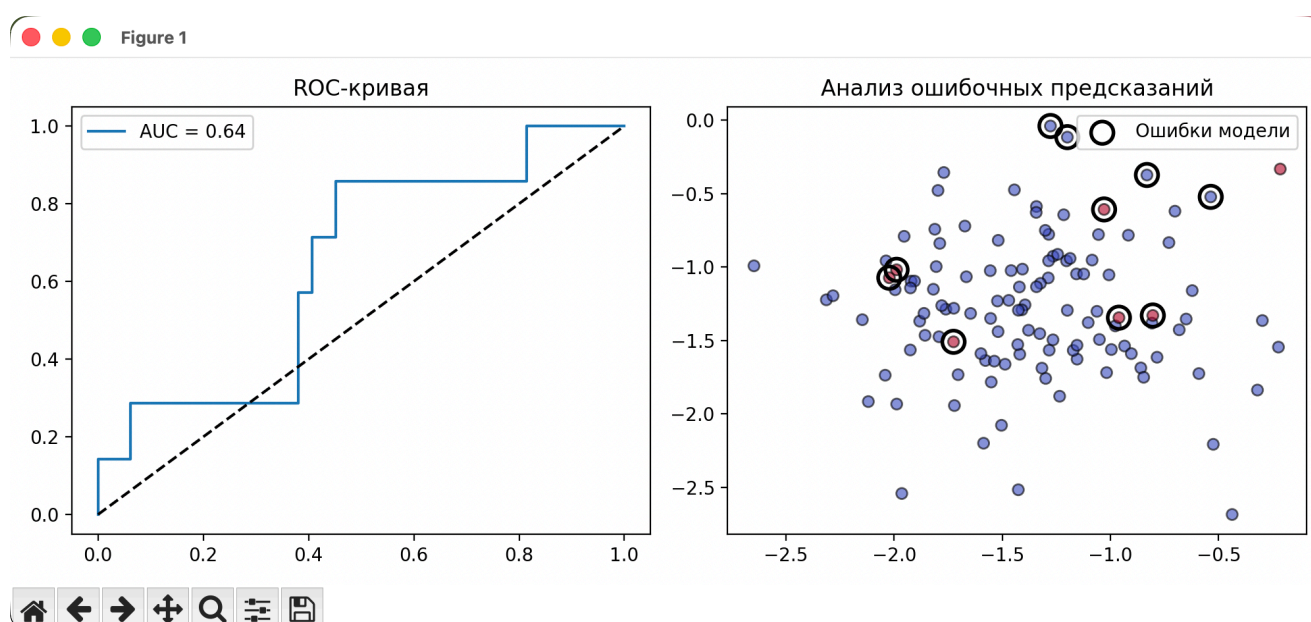
Был реализован Hinge Loss ($y \in \{-1, +1\}$) и добавлен расчет штрафа за норму весов L_2 .



Вывод: С ростом коэффициента регуляризации λ норма весов $\|w\|_2$ предсказуемо падает. Жесткое ограничение весов помогает бороться с переобучением, однако слишком большие значения ($\lambda = 1.0$) приводят к деградации обобщающей способности (underfitting), так как веса стремятся к нулю.

6.3. Метрики качества и анализ ошибок

Помимо метрики Ассигасу, были добавлены метрики: **Precision, Recall, F1-score** и **ROC-AUC**. Построена ROC-кривая, визуализирующая соотношение TPR и FPR при изменении порога вероятности.

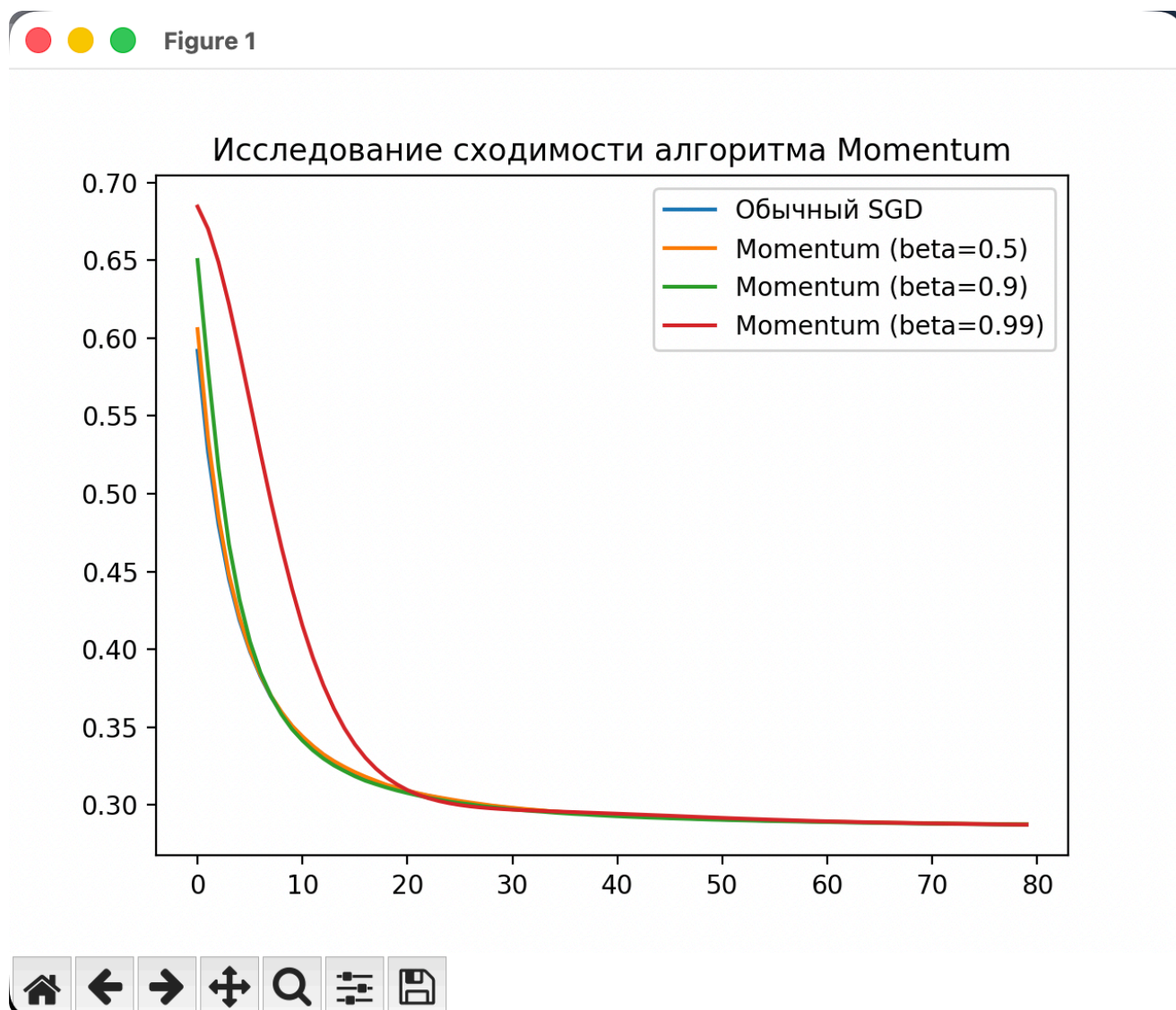


6.4. Исследование сходимости с Momentum

В класс перцептрона внедрен градиентный спуск с импульсом (Momentum SGD):

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla L$$

$$w = w - \eta v_t$$



Вывод: Добавление импульса ($\beta = 0.5, 0.9$) существенно сглаживает осцилляции при обучении мини-батчами и позволяет быстрее спускаться в локальный минимум, сохраняя инерцию движения вдоль оврагов функции потерь.

6.5. Кросс-валидация и подбор гиперпараметров (Grid Search)

Для автоматического поиска лучших гиперпараметров была реализована функция 5-fold (5-кратной) кросс-валидации.

Была протестирована сетка гиперпараметров:

- Скорости обучения (`lr`): 0.01, 0.1, 0.5
- Размеры батча (`batch_size`): 16, 32, 64

По результатам кросс-валидации лучшей комбинацией оказалась: **LR = 0.01, Batch Size = 64** со средней точностью на валидации **~0.943**. Финальная модель, обученная с этими параметрами, показала высокую устойчивость на тестовой выборке.

Заключение

В ходе лабораторной работы был успешно реализован и обучен однослойный перцептрон без использования сторонних фреймворков глубокого обучения. Было проведено исследование влияния гиперпараметров, что позволило лучше понять их роль в сходимости градиентного спуска.

Выполнение бонусной части доказало ключевые недостатки модели (неспособность решать нелинейные задачи вроде XOR) и способы её улучшения (L2-регуляризация, Momentum SGD, валидация по метрикам ROC-AUC). Работа позволила закрепить математическую базу нейронных сетей и навыки векторизации кода.